

Experiment 10: Input & Output Manual

Integration of SSL Certificate in Web Application

🎓 Student & Laboratory Credentials

- **Student Name:** Asmit Jogdand
- **Roll Number:** 25CE1051
- **Class / Division:** B.Tech Computer Engineering (Sem VI)
- **Academic Year:** 2026 – 2027
- **Course:** Skill Based Lab - Advanced Web Technology (SBL-AWT)
- **Institution:** Ramrao Adik Institute of Technology (RAIT), D Y Patil Deemed to be University, Nerul, Navi Mumbai
- **Problem Statement Domain:** HealthPulse — Digital Healthcare & Clinical Management Portal
- **Experiment Title:** Integration of SSL Certificate in Web Application

1. Experiment Aim

To configure, integrate, and verify end-to-end cryptographic transport security (**SSL/TLS**) within the web application using OpenSSL X.509 digital certificates, a Node.js native HTTPS server enforcing modern ciphers (TLS 1.2 / TLS 1.3), automated HTTP-to-HTTPS 301 redirection, and an Nginx reverse proxy configuration.

2. 📁 Input Specification

2.1 File System Input

File Name	Format	Role & Implementation Responsibility
generate-ssl.sh	Bash Shell Script	Automates OpenSSL private key (RSA 2048-bit) and X.509 self-signed certificate generation with Subject Alternative Names (SANs).
san.cnf	OpenSSL Config	Defines certificate authority extensions and Subject Alternative Names (localhost , healthpulse-care.local , 127.0.0.1).
server-key.pem	PEM (Base64)	2048-bit RSA Private Key used by the HTTPS server for TLS session negotiation.
server-cert.pem	PEM (Base64)	X.509 Public Key Certificate signed by HealthPulse Clinical Authority.
server-https.js	Node.js / Express	Dual-server implementation: HTTPS Server on Port 8443 and HTTP Redirection server on Port 8080 with HSTS security headers.
nginx-ssl.conf	Nginx Config	Production reverse proxy configuration terminating SSL and forwarding traffic to upstream Node clusters.
package.json	JSON	Project dependencies (express) and launch scripts.

2.2 Execution Command Input

```
# Navigate to the experiment directory
cd /home/ubuntu/SBL-EXPT/Asmit_25CE1051/Expt-10-SSL-Certificate

# Step 1: Generate private keys and X.509 certificates
bash generate-ssl.sh

# Step 2: Install dependencies
npm install

# Step 3: Launch Dual Server (HTTPS Port 8443 + HTTP Redirect Port 8080)
node server-https.js

# Custom port execution (if needed):
HTTPS_PORT=8443 HTTP_PORT=8080 node server-https.js
```

2.3 Cryptographic Certificate Parameters (Input)

```
+-----+
| PARAMETER NAME      | VALUE ASSIGNED / SPECIFICATION |
+-----+
| Key Algorithm       | RSA (Rivest-Shamir-Adleman)    |
| Key Size            | 2048 bits                      |
| Signature Hash      | SHA-256 (Secure Hash Algorithm 256-bit) |
| Certificate Validity| 365 Days                      |
| Common Name (CN)   | localhost                     |
| Organization (O)    | HealthPulse Super-Specialty Clinic |
| Organizational Unit(OU)| Computer Engineering Dept, RAIT |
| City / Locality (L) | Navi Mumbai                   |
| State (ST)          | Maharashtra                    |
| Country (C)         | IN (India)                    |
| Subject Alt Names   | DNS:localhost, DNS:healthpulse-care.local, IP:127.0.0.1 |
| Enforced Protocols  | TLSv1.2, TLSv1.3 (SSLv2, SSLv3, TLSv1.0, TLSv1.1 disabled) |
+-----+
```

2.4 Client Test Commands (Input)

```
# Test 1: Verify HTTP to HTTPS 301 Redirection
curl -I http://localhost:8080/

# Test 2: Verify Encrypted HTTPS Handshake and Query SSL Telemetry Endpoint
curl -k -s https://localhost:8443/api/ssl-info

# Test 3: Inspect Local X.509 Certificate Details via OpenSSL
openssl x509 -in server-cert.pem -text -noout | grep -E "(Issuer|Subject|Not After|DNS:)"
```

3. Output Specification

3.1 OpenSSL Certificate Generation Output (`generate-ssl.sh`)

```
$ bash generate-ssl.sh
[HealthPulse OpenSSL] Initializing 2048-bit RSA Private Key generation...
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
writing RSA key to 'server-key.pem'
[HealthPulse OpenSSL] Signing X.509 Certificate with SAN extensions...
Signature ok
subject=C = IN, ST = Maharashtra, L = Navi Mumbai, O = HealthPulse Super-Specialty Clinic, OU = Computer Engineering Dept, RAIT, CN = localhost
Getting CA Private Key
[HealthPulse OpenSSL] Verifying certificate cryptographic integrity...
server-cert.pem: OK
[HealthPulse OpenSSL] SSL Certificates successfully provisioned in /Expt-10-SSL-Certificate.
```

3.2 Dual Server Terminal Execution Logs

```
$ HTTPS_PORT=8443 HTTP_PORT=8080 node server-https.js
[HealthPulse HTTPS] Secure server active on https://localhost:8443
[HealthPulse HTTP Redirect] Port 8080 redirecting to HTTPS Port 8443
[HealthPulse Author] Asmit Jogdand (25CE1051)
[HTTP Redirect] 301 Redirecting / -> https://localhost:8443/
[TLS Handshake] Negotiated Protocol: TLSv1.3 | Cipher: TLS_AES_256_GCM_SHA384
```

3.3 HTTP to HTTPS 301 Permanent Redirection Output (`curl -I http://localhost:8080/`)

```
HTTP/1.1 301 Moved Permanently
X-Powered-By: Express
Location: https://localhost:8443/
Vary: Accept
Content-Type: text/plain; charset=utf-8
Content-Length: 57
Date: Thu, 01 Oct 2026 09:32:23 GMT
Connection: keep-alive
```

3.4 Encrypted SSL Telemetry API Output (`GET https://localhost:8443/api/ssl-info`)

```
HTTP/1.1 200 OK
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Content-Type: application/json; charset=utf-8

{
  "httpsEnabled": true,
  "tlsProtocol": "TLSv1.3",
  "cipherSuite": "TLS_AES_256_GCM_SHA384",
  "hstsActive": true,
  "keyLength": "2048 bits",
  "subject": {
    "commonName": "localhost",
    "organization": "HealthPulse Super-Specialty Clinic",
    "department": "Computer Engineering Dept, RAIT",
    "location": "Navi Mumbai, Maharashtra, India"
  },
  "student": "Asmit Jogdand (25CE1051)",
  "timestamp": "2026-10-01T09:32:23.735Z"
}
```

3.5 Graphical User Interface Output Representation (<https://localhost:8443/>)

```

+-----+
| 0 https://localhost:8443/ [Connection Secure - TLS 1.3] Student: Asmit (25CE1051) |
+-----+
|
|      ▫ HEALTHPULSE CRYPTOGRAPHIC TRANSPORT SECURITY HUB
|      End-to-End Encrypted Healthcare & HIPAA-Compliant HTTPS Portal
|
| +-- Real-Time Cryptographic Handshake Telemetry -----+
| | Encryption Status: [ ● ACTIVE & ENCRYPTED (HTTPS) ] |
| | Negotiated TLS Protocol: TLSv1.3 (Modern RFC 8446 Specification) |
| | Cipher Suite: TLS_AES_256_GCM_SHA384 (Galois/Counter Mode 256-bit) |
| | Asymmetric Key: RSA 2048-bit Private/Public Keypair |
| | Subject Alternative Name: localhost, healthpulse-care.local, 127.0.0.1 |
| +-----+
|
| +-- Active Enterprise Security Headers -----+
| | [✓] Strict-Transport-Security (HSTS): max-age=31536000; includeSubDomains; preload |
| | [✓] X-Content-Type-Options: nosniff (Prevents MIME-type sniffing attacks) |
| | [✓] X-Frame-Options: SAMEORIGIN (Mitigates clickjacking overlays) |
| | [✓] HTTP 301 Redirect: Automatic port 8080 traffic upgrade to secure port 8443 |
| +-----+
|
| +-- Production Nginx Reverse Proxy Architecture -----+
| | [ Client Browser ] --- (Port 443 / SSL) ----> [ Nginx Proxy ] --- (Port 3000) ----> [ Node ] |
| +-----+

```

3.6 Test Case Execution Results

Test Scenario	Action Injected	Server & Cryptographic Execution	Observed Result Output	Status
Cert Generation	<code>bash generate-ssl.sh</code>	OpenSSL generates RSA key + cert	Creates <code>server-key.pem</code> and <code>server-cert.pem</code>	PASS
SAN Verification	Inspect certificate	OpenSSL parses X.509 v3 extensions	Confirms SANs: <code>localhost</code> , <code>127.0.0.1</code>	PASS
HTTP Redirection	<code>curl -I http://localhost:8080/</code>	HTTP server intercepts port 8080	Returns HTTP 301; redirects to <code>https://...:8443</code>	PASS
HTTPS Telemetry	<code>curl -k https://localhost:8443/api/ssl-info</code>	Node HTTPS parses socket cipher	Returns TLS 1.3 <code>TLS_AES_256_GCM_SHA384</code>	PASS
HSTS Enforcement	Inspect HTTP response headers	Middleware sets security headers	<code>Strict-Transport-Security: max-age=31536000</code>	PASS
Clickjacking Guard	Inspect HTTP response headers	Sets <code>X-Frame-Options</code>	Value <code>SAMEORIGIN</code> confirmed in header	PASS

4. Conclusion & Verification

Experiment 10 successfully integrated enterprise SSL/TLS encryption. OpenSSL generated a 2048-bit RSA keypair and SAN-compliant certificate, Node.js executed an encrypted HTTPS server with automated HTTP 301 redirection, and strict transport security (HSTS) headers safeguarded against man-in-the-middle attacks.